

B0911007H-01/02 2025-2026学年春季学期

计算机组成原理(研讨课)

E03 Verilog HDL硬件描述语言基础

潘宇轩、林一晖

2026年3月12日



中国科学院大学
University of Chinese Academy of Sciences



中科院计算所
INSTITUTE OF COMPUTING TECHNOLOGY, CAS

基本设计思想——描述电路



定义：Verilog是一种用于描述、设计电子系统（特别是数字电路）的硬件描述语言

写Verilog最根本的目的，就是描述出一个逻辑功能正确的，可综合的电路。

可以在不同的抽象层次上描述电路

门级：

```
module full_adder (  
    input A,           // 输入A  
    input B,           // 输入B  
    input Cin,         // 低位进位输入  
    output Sum,        // 和  
    output Cout        // 进位输出  
);  
  
    // 中间信号  
    wire S1, C1, C2;  
  
    // 和的计算  
    xor(S1, A, B);      // A XOR B  
    xor(Sum, S1, Cin);  // (A XOR B) XOR Cin  
  
    // 进位的计算  
    and(C1, A, B);      // A AND B  
    and(C2, S1, Cin);   // (A XOR B) AND Cin  
    or(Cout, C1, C2);   // (A AND B) OR ((A XOR B) AND Cin)
```

endmodule

行为级：

```
module full_adder (  
    input A,  
    input B,  
    input Cin,  
    output Sum,  
    output Cout  
);  
  
    // 三个输入中奇数个为1时，和为1  
    assign Sum = A ^ B ^ Cin;  
  
    // 任意两个输入为1时，产生进位  
    assign Cout = (A & B) | (B & Cin) | (A & Cin);  
  
endmodule
```

行为级抽象层次更高，更利于理解。在之后的实验中都使用行为级描述。

基本设计单元——模块



封装模块的关键字

```
module 模块名(  
    input|output <reg|wire> [信号位宽-1:0] 端口名1,  
    ...  
    input|output <reg|wire> [信号位宽-1:0] 端口名N  
); //别忘了分号☺
```

端口列表

```
    reg [信号位宽-1:0] R变量1,R变量2...;  
    wire [信号位宽-1:0] W变量1,W变量2...;
```

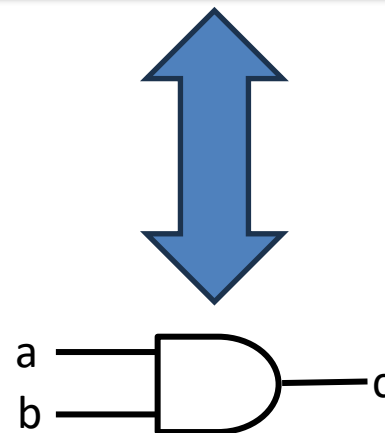
内部信号说明

```
    模块名 实例名(.端口名M(W变量1)...);  
    assign W变量 = 表达式;  
    always @(敏感列表) begin  
        R变量 <= 表达式;  
        ...  
    end
```

逻辑功能定义

```
endmodule
```

```
module my_and(  
    input a,b,  
    output c  
);  
    and and1(c,a,b);  
endmodule
```



数据类型和表示 (1)



- ❑ wire型变量：硬件单元的物理连线
- ❑ reg型变量：数据存储单元的抽象
- ❑ memory型变量：reg型变量数组，对存储器的建模
- ❑ 模块端口默认为wire类型
 - 模块描述时，input端口只能定义为wire类型，output可以定义为wire/reg类型
 - 模块例化时，input端口可以为wire/reg类型，output只能为wire类型

~~❑ integer型、parameter型、tril型、trio型、wand型.....~~

```
wire [31:0] data; //32位wire型变量
reg [15:0] status; //16位reg型变量
reg [15:0] regfile [31:0]; //32个16位reg型变量构成的memory型变量
```

<-注意向量位宽和数组大小的声明方式

数据类型和表示 (2)



□ 四种基本的电平逻辑

- 0: 逻辑0
- 1: 逻辑1
- X: 不定态
- Z: 高阻态

```
===== Testbench Log Start =====  
VCD info: dumpfile fpga/sim_out/example/dump.vcd opened for output.  
ERROR: 0x01 + 0x00, golden result: 0x01, current result: 0xzz.  
===== Testbench Log Ended =====
```



□ 整数数值的表示方法

- <位宽>,<进制符号><数值>

```
wire [31:0]counter;  
assign counter = 32'b100; //二进制表示  
assign counter = 32'h64; //十六进制表示  
assign counter = 32'd100; //十进制表示  
assign counter = 'd100; //EDA一般会自动分辨位宽  
assign counter = 32'h3200_c0de; //下划线增强代码可读性
```



```
wire [31:0]counter;  
assign counter = 'bx; //不要出现在代码中出现不定态和高阻态!
```

□ 连续赋值

- 赋值在输入发生改变时马上发生
- 使用assign语句
- 在initial或always代码块外使用
- 针对wire类型

□ 过程赋值

- 只有敏感列表条件触发时才发生
- 可在always代码块内使用
- 组合逻辑用变量 = 表达式赋值
 - 仅在三段式状态机的第二段描述中
 - 实验项目三详细介绍
- 时序逻辑用变量 <= 表达式赋值
- 针对reg类型

assign语句格式:

assign 变量名 = 表达式;

always代码块格式:

```
always @(敏感列表) begin
    变量名 <= 表达式; end
```

@(switch0)表示switch0电平改变

@(*)表示任何输入变化

@(posedge clk)表示clk上升沿，不要混用上升/下降沿

连续赋值示例

```
wire led;
assign led = switch0 & switch1;
```

过程赋值示例

```
reg led, led_reg;
always @(*) begin //组合逻辑
    led = (switch0 & switch1);
end
always @(posedge clk) begin //时序逻辑
    led_reg <= led;
end
```

运算符



□ 位拼接运算符

- 格式: {信号1某几位, 信号2某几位...}
- 把多个信号的某几位拼起来进行运算

□ 重复运算符

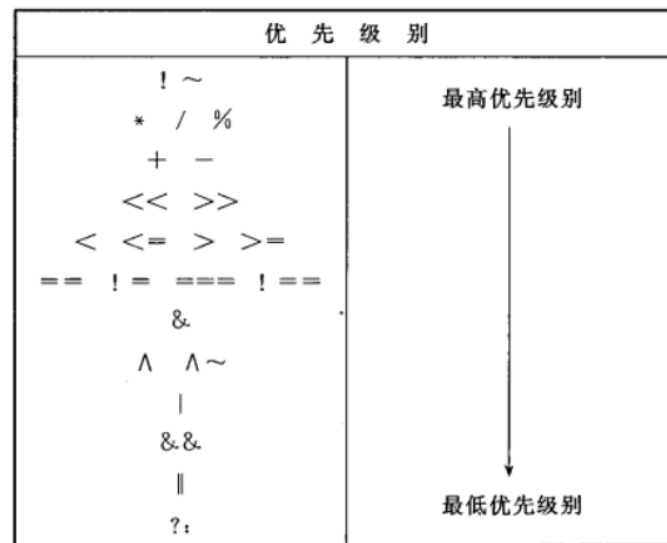
- 格式: {重复次数{信号的某几位}}
- 可以将同一个操作数拼接多次

□ 条件运算符

- 格式: 条件表达式 ? 真表达式 : 假表达式
- 会被综合成一个二路选择器

□ 归约操作符

- 格式: (| or &)信号
- 会用信号的每一位计算出一个结果



```
//将结果高位赋值给cout, 低位赋值给sum
assign {cout,sum}=A+B+cin;
//重复data四遍赋值给rep_data
assign rep_data[63:0]={4{data[15:0]}};
//将要rep_data, last信号拼接, 赋值给bits
assign bits[64:0]={rep_data,last};
//当A等于3时, out赋值为x, 否则为y
assign out=(A==3)?x:y;
//当A!=0时, B=1; 当A==0时, B=0
assign B = |A
```

□ 乘法 (*)

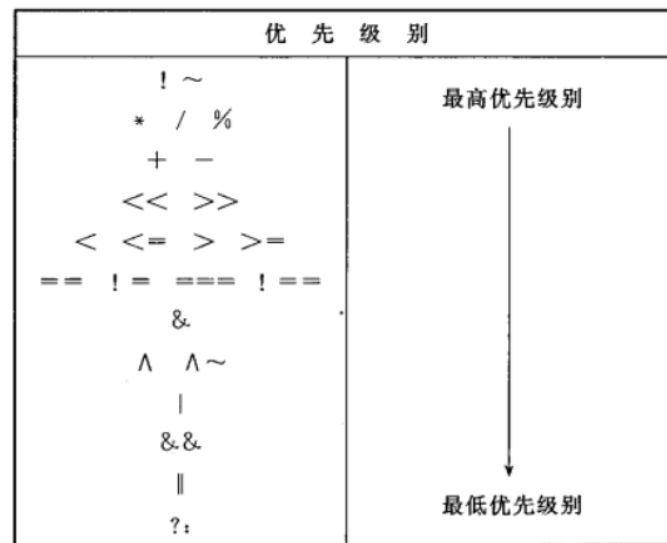
- 可以直接使用，综合时会自动选择合适的乘法实现
- 两个乘数是a位时，得数需要被扩展至2a位
- 可以使用\$signed或者\$unsigned修饰有符号/无符号

□ 除法/求余 (/ or %)

- 不能直接使用，一般需要多拍完成
- 本课程不涉及除法器

□ 算数/逻辑右移

- 推荐使用拼接高位的方式完成
- 可以使用>>>算符进行算数右移



```
//乘法
assign xy = x * y;
//逻辑右移
assign a = b >> c;
//算数右移
assign a = b >>> c;
```


组合逻辑——以半加器和全加器为例（1）



□ 半加器：没有低位进位、有高位进位的加法器

示例1

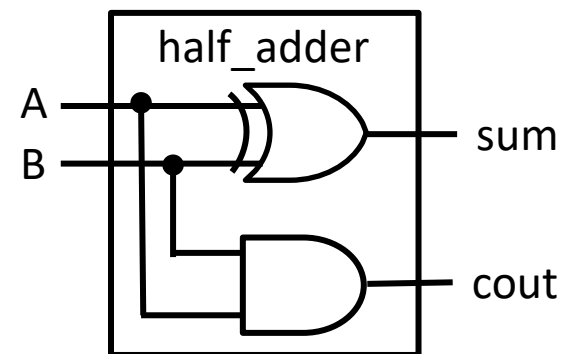
```
module half_adder(  
    input A,B,  
    output sum,cout  
);  
    xor xor1(sum,A,B);  
    and and1(cout,A,B);  
endmodule
```

示例2

```
module half_adder(  
    input A,B,  
    output sum,cout  
);  
    assign sum=A^B;  
    assign cout=A&B;  
endmodule
```

连续赋值，当输入发生改变时马上发生

注意赋值符号



半加器电路图

A	B	sum	cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

半加器真值表

组合逻辑——以半加器和全加器为例 (2)



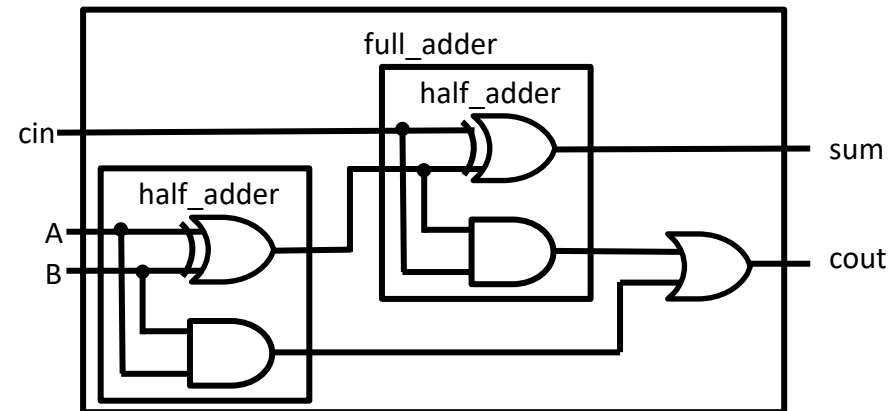
□ 全加器：有高位进位和低位进位的加法器

```
module full_adder(//利用半加器实现全加器
    input A,B,cin,
    output sum,cout
);
    wire w1,w2,w3; //声明三个内部wire信号
    half_adder add1(.A(A),.B(B),.sum(w1),.cout(w2));
    half_adder add2(.A(cin),.B(w1),.sum(sum),.cout(w3));
    assign cout = w2|w3;
endmodule
```

half_adder模块的两个实例

```
module full_adderV2(//利用+运算符和位拼接运算符实现全加器
    input A,B,cin,
    output sum,cout
);
    assign {cout,sum} = A + B + cin;
endmodule
```

拼接成两位变量
综合成全加器逻辑



全加器电路图

A	B	cin	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

全加器真值表

时序逻辑——以D-触发器为例



- D-触发器：在时钟边沿到来时，存储输入值的寄存器
- 在一个时钟周期内计算，在时钟沿到来时存进寄存器

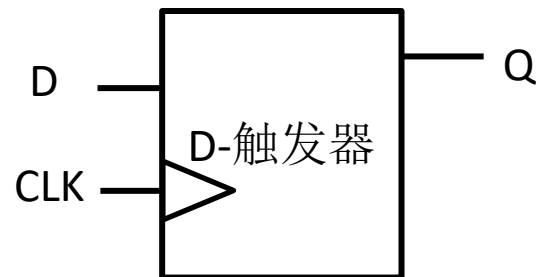
示例

```
module d_flip_flop(  
    input d,clk,  
    output reg q);  
    always @(posedge clk)  
    begin  
        q <= d; // 上升沿有效的时候，把d捕获到q  
    end  
endmodule
```

过程赋值，posedge clk
表示下列逻辑只有在时钟
上升沿时工作

“always @(posedge clk)”
是综合出触发器的关键

注意赋值符号



D-触发器电路图

D	Q _n	Q _{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

D-触发器真值表

- ❑ Verilog代码中经常会有同一个格式重复多次的情况，这时最好换行，并对齐上下的变量和符号
- ❑ 除开单目运算符，在运算符和变量间空一格
- ❑ 使用Tab控制缩进
- ❑ 做好关键信号、电路逻辑和部件结构的注释
- ❑ 最终目的是使代码清晰，不仅自己可读，也能让其他人可读

```
// inst sram interface
output wire      inst_sram_req,
output wire      inst_sram_wr,
output wire [1 :0] inst_sram_size,
output wire [3 :0] inst_sram_wstrb,
output wire [31:0] inst_sram_addr,
output wire [31:0] inst_sram_wdata,
output wire      inst_sram_cached,
input  wire      inst_sram_addr_ok,
input  wire      inst_sram_data_ok,
input  wire [31:0] inst_sram_rdata,
```

```
assign IF_nextpc_calc = {32{~WB_set_invalid      }} & WB_ex_entry
                        |{32{ WB_set_invalid      & ID_br_taken      }} & ID_br_target
                        |{32{ WB_set_invalid      & ~ID_br_taken     }} & IF_pc+32'h4;
assign IF_nextpc      = {32{~IF_pc_has_changed    }} & IF_nextpc_calc
                        |{32{ IF_pc_has_changed & WB_set_invalid    }} & IF_changed_pc
                        |{32{ IF_pc_has_changed & ~WB_set_invalid   }} & IF_nextpc_calc;
assign IF_to_IDinst   = {32{ IF_state[DATA_OKI]&~IF_ex_adeft_next}} & inst_sram_rdata
                        |{32{ IF_state[WAIT_IDI]&~IF_ex_adeft_next}} & IF_keepinst;
```

注：图片所示内容和课程实验内容无关，仅作参考

切勿在设计中使用Initial语句



```
module timer (  
    input  wire  clk,  
    input  wire  reset,  
    output reg [7:0] my_register  
);  
    initial begin  
        my_register <= 8'b00000000;  
    end  
  
    always @(posedge clk) begin  
        my_register <= my_register + 1;  
    end  
endmodule
```



```
module timer (  
    input  wire  clk,  
    input  wire  reset,  
    output reg [7:0] my_register  
);  
    always @(posedge clk) begin  
        if (reset) begin  
            my_register <= 8'b00000000;  
        end else begin  
            my_register <= my_register + 1;  
        end  
    end  
endmodule
```



- initial只能在testbench中使用
- 它不会被综合成逻辑电路，不可在设计代码中使用

切勿在设计中使用wait语句和延迟语句



```
module b_after_a (  
    input  clk,  
    input  rst,  
    input  singal,  
    output a,  
    output b  
);  
    always @(posedge clk) begin  
        if(rst)begin  
            a <= 0;  
            b <= 1;  
        end else begin  
            a <= ~a;  
            #10  
            wait(signal==1)  
            b <= ~b;  
        end  
    end  
endmodule
```



- 调试阶段观察波形后发现，如果某些信号的值能晚一点变化逻辑就正确了
- 于是就在设计中使用了wait语句和延迟语句（以#开始），发现波形正确
- 但这些语句和前面的initial语句一样，无法被综合为逻辑电路

• 再次强调

- 写Verilog HDL进行电路设计的目的是**描述**出一个**逻辑功能正确且可综合的电路**
- **不要使用某些高级语法或不可综合语句**，描述出一个仅可仿真通过的逻辑功能

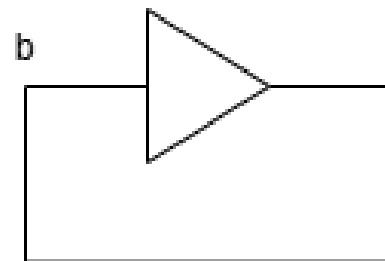
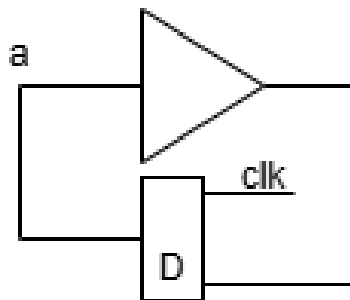
切勿在设计中出现组合逻辑环



```
module invert_a_and_b (  
    input  wire clk,  
    input  wire rst,  
    output reg a,  
    output reg b  
);  
    always @(posedge clk) begin  
        if(rst)begin  
            a <= 0;  
        end else begin  
            a <= ~a;  
        end  
    end  
    always @(*) begin  
        b = ~b;  
    end  
endmodule
```



- 左边代码示例中，变量a和变量b的取值变化都和自身取值相关，但对应的电路结构不同



- 变量b赋值语句的描述会出现组合逻辑环
 - 即组合逻辑电路的输出在不经过任何时序逻辑电路（如寄存器）的情况下，直接反馈到该电路所在组合逻辑路径中更早出现的电路输入端
- 组合逻辑环的危害
 - 逻辑行为仿真正确，但EDA工具生成的硬件电路不能正常工作
 - 无法确定组合逻辑电路的延时，导致时序无法收敛

□ 练习网站

- HDLBits: <https://hdlbits.01xz.net/>
- 中科大Vlab Verilog OJ: <https://verilogoj.ustc.edu.cn/oj/>

□ 参考网站

- IEEE verilog standard
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1620780>
- 菜鸟教程: <https://www.runoob.com/w3cnote/verilog-tutorial.html>

□ 参考书籍

- 《Verilog数字系统设计教程》，夏宇闻编著，北京航空航天大学出版社
- 《Verilog FPGA芯片设计》，林灶生、刘绍汉编著，北京航空航天大学出版社

□ 下列哪个32位寄存器的代码是正确的？

```
module Register (  
    input wire clk,  
    input wire enable,  
    input wire [31:0] data_in,  
    output reg [31:0] data_out  
);  
    initial begin  
        data_out = 32'b0;  
    end  
    always @(posedge clk) begin  
        if (enable) begin  
            data_out = data_in;  
        end  
    end  
  
endmodule
```

A

```
module Register (  
    input wire clk,  
    input wire reset,  
    input wire enable,  
    input wire [31:0] data_in,  
    output reg [31:0] data_out  
);  
  
    always @(reset) begin  
        if (reset) begin  
            data_out <= 32'b0;  
        end else begin  
            if (enable) begin  
                data_out <= data_in;  
            end  
        end  
    end  
  
endmodule
```

B

```
module Register (  
    input wire clk,  
    input wire reset,  
    input wire enable,  
    input wire [31:0] data_in,  
    output reg [31:0] data_out  
)  
  
    always @(posedge clk) begin  
        if (reset) begin  
            data_out <= 32'b0;  
        end else begin  
            if (enable) begin  
                data_out <= data_in;  
            end  
        end  
    end  
  
endmodule
```

C

□ 下列哪个32位寄存器的代码是正确的？

```
module Register (  
    input wire clk,  
    input wire enable,  
    input wire [31:0] data_in,  
    output reg [31:0] data_out  
);  
    initial begin  
        data_out = 32'b0;  
    end  
    always @(posedge clk) begin  
        if (enable) begin  
            data_out = data_in;  
        end  
    end  
  
endmodule
```

A

```
module Register (  
    input wire clk,  
    input wire reset,  
    input wire enable,  
    input wire [31:0] data_in,  
    output reg [31:0] data_out  
);  
  
    always @(reset) begin  
        if (reset) begin  
            data_out <= 32'b0;  
        end else begin  
            if (enable) begin  
                data_out <= data_in;  
            end  
        end  
    end  
  
endmodule
```

B

```
module Register (  
    input wire clk,  
    input wire reset,  
    input wire enable,  
    input wire [31:0] data_in,  
    output reg [31:0] data_out  
); //别忘了分号☺  
  
    always @(posedge clk) begin  
        if (reset) begin  
            data_out <= 32'b0;  
        end else begin  
            if (enable) begin  
                data_out <= data_in;  
            end  
        end  
    end  
  
endmodule
```

C

問 与 答



中国科学院大学
University of Chinese Academy of Sciences



中科院计算所
INSTITUTE OF COMPUTING TECHNOLOGY, CAS